

Step 1: Domain Model & In-Memory Data Layer

Core Object-Oriented Modeling, Encapsulation, & Thread-Safe State Management

Course: Advanced Java Enterprise Engineering (EILMS Project) | **Module:** Domain Engineering & Core Java Architecture

1. Executive Summary & Architectural Context

In enterprise software engineering, before connecting to external database engines or exposing REST/SOAP web services, applications must establish a rigorous, well-encapsulated **Domain Layer**. Step 1 of the Enterprise Inventory & Logistics Management System (EILMS) focuses on building clean Java objects, data transfer representations, and a high-throughput, thread-safe in-memory repository.

This foundation guarantees that business invariants, validation rules, and concurrent state modifications are verified directly inside JVM memory before introducing persistence frameworks.

2. Maven Project & Core Module Setup

Step 1 utilizes standard Java Core libraries without heavy external framework dependencies, laying down the core project layout (`eilms-core`) and compilation settings in `pom.xml`.

```
<!-- pom.xml Base Project Configuration -->
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/
maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.eilms</groupId>
  <artifactId>eilms-core</artifactId>
  <version>1.0.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
</project>
```

PROFESSOR'S PEDAGOGICAL ANALYSIS: ARCHITECTURE BASELINE

- **Decoupled Design:** By omitting external dependencies in Step 1, the core domain model remains pure Java (POJO), ensuring high unit-testability and seamless portability.
- **Java Standard Specs:** Target version set to Java 17 LTS to utilize modern language features while maintaining enterprise compatibility.

3. Implementation Details & Pedagogical Code Walkthrough

File 1: `src/main/java/com/eilms/model/WarehouseItem.java` (Domain Object)

The central domain model enforcing business constraints, encapsulation, and identity contracts.

```
package com.eilms.model;

import java.util.Objects;
```

```

public class WarehouseItem {
    private String id;
    private String name;
    private int quantity;

    public WarehouseItem() {}

    public WarehouseItem(String id, String name, int quantity) {
        if (quantity < 0) {
            throw new IllegalArgumentException("Stock quantity cannot be negative.");
        }
        this.id = id;
        this.name = name;
        this.quantity = quantity;
    }

    // Encapsulated Getters & Setters
    public String getId() { return id; }
    public void setId(String id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public int getQuantity() { return quantity; }
    public void setQuantity(int quantity) {
        if (quantity < 0) {
            throw new IllegalArgumentException("Stock quantity cannot be negative.");
        }
        this.quantity = quantity;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        WarehouseItem that = (WarehouseItem) o;
        return Objects.equals(id, that.id);
    }

    @Override
    public int hashCode() {
        return Objects.hash(id);
    }
}

```

PROFESSOR'S EXPLANATION: WHY WE CREATED THIS CODE

- **Encapsulation & Invariants:** Stock quantity validation directly inside constructors and setters prevents illegal negative inventory states.
- **Identity Contract:** Implements `equals()` and `hashCode()` based strictly on the unique `id` field, enabling safe usage inside Java Collection frameworks (`HashSet`, `HashMap`).

File 2: `src/main/java/com/eilms/repository/InMemoryInventoryRepository.java` (Thread-Safe Repository)

An in-memory persistence layer managing inventory data concurrently across multithreaded runtime requests.

```

package com.eilms.repository;

import com.eilms.model.WarehouseItem;
import java.util.*;

```

```
import java.util.concurrent.ConcurrentHashMap;

public class InMemoryInventoryRepository {
    // ConcurrentHashMap ensures lock-free / fine-grained thread-safe access
    private final Map storage = new ConcurrentHashMap<>();

    public void save(WarehouseItem item) {
        if (item == null || item.getId() == null) {
            throw new IllegalArgumentException("Item and Item ID must not be null.");
        }
        storage.put(item.getId(), item);
    }

    public Optional findById(String id) {
        return Optional.ofNullable(storage.get(id));
    }

    public List findAll() {
        return new ArrayList<>(storage.values());
    }

    public void deleteById(String id) {
        storage.remove(id);
    }

    public void clear() {
        storage.clear();
    }
}
```

PROFESSOR'S EXPLANATION: WHY WE CREATED THIS CODE

- **Thread Safety via `ConcurrentHashMap`:** Standard `HashMap` is unsafe under concurrent web server operations. `ConcurrentHashMap` uses bucket-level locking to achieve high concurrency without global thread blocking.
- **Null Safety via `Optional`:** Returning `Optional` explicitly forces client callers to handle missing items gracefully, eliminating potential `NullPointerException` bugs.

File 3: `src/main/java/com/eilms/Step1TestMain.java` (Verification Program)

A executable driver class verifying repository operations and memory management.

```
package com.eilms;

import com.eilms.model.WarehouseItem;
import com.eilms.repository.InMemoryInventoryRepository;

public class Step1TestMain {
    public static void main(String[] args) {
        InMemoryInventoryRepository repository = new InMemoryInventoryRepository();

        // 1. Populate initial in-memory items
        WarehouseItem item1 = new WarehouseItem("ITEM-101", "Core Gateway Router", 50);
        WarehouseItem item2 = new WarehouseItem("ITEM-102", "Fiber Optic Transceiver", 120);

        repository.save(item1);
        repository.save(item2);

        System.out.println(">>> Initial Inventory Saved:");
        repository.findAll().forEach(item ->
            System.out.println("ID: " + item.getId() + " | Name: " + item.getName() + " | Qty: " +
            item.getQuantity())
        );
    }
}
```

```
// 2. Query Item by ID using Optional
repository.findById("ITEM-101").ifPresent(item -> {
    System.out.println("
>>> Retrieved Item ITEM-101: " + item.getName());
});
}
```

PROFESSOR'S EXPLANATION: WHY WE CREATED THIS CODE

- **Functional Processing:** Demonstrates modern Java lambda operations (`forEach`, `ifPresent`) for clean state interaction.
- **Execution Baseline:** Verifies that in-memory CRUD operations function deterministically prior to integrating relational database drivers in Step 2.

4. Execution & Memory State Verification

Executing `Step1TestMain.java` produces structured log output detailing successful in-memory entity management:

```
>>> Initial Inventory Saved:
ID: ITEM-101 | Name: Core Gateway Router | Qty: 50
ID: ITEM-102 | Name: Fiber Optic Transceiver | Qty: 120

>>> Retrieved Item ITEM-101: Core Gateway Router
```

Item ID	Item Name	Quantity	Memory Repository Status
ITEM-101	Core Gateway Router	50	Stored (`ConcurrentHashMap`)
ITEM-102	Fiber Optic Transceiver	120	Stored (`ConcurrentHashMap`)

End of Architectural Lecture Guide — Step 1 Domain Model & In-Memory Layer